EECS 836: Machine Learning

Zijun Yao

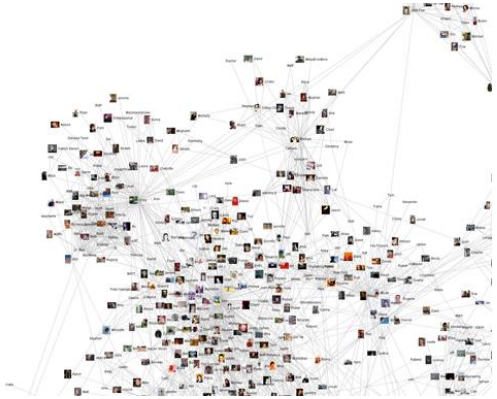
Assistant Professor, EECS Department

The University of Kansas

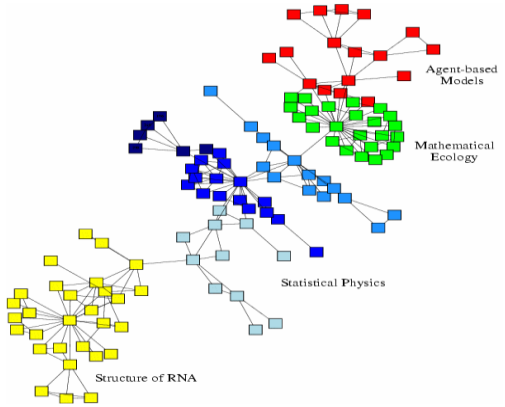
Agenda

- Graph Neural Networks (GNN)
  - What is graph learning
  - Neighborhood aggregation as message passing
  - Graph convolutional network (GCN)

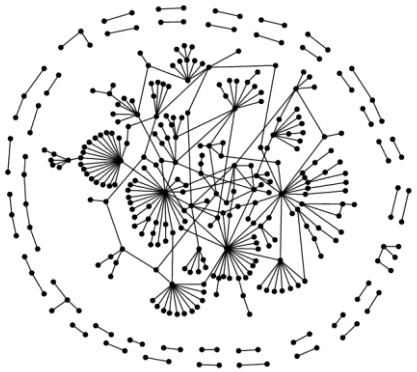
Many Data are networks



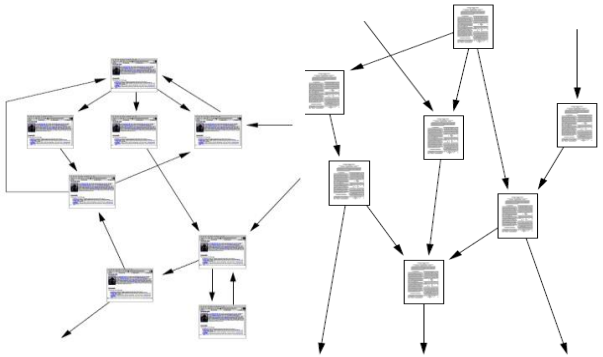
Social networks



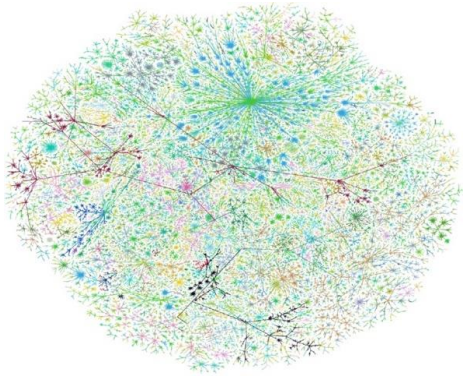
Economic networks



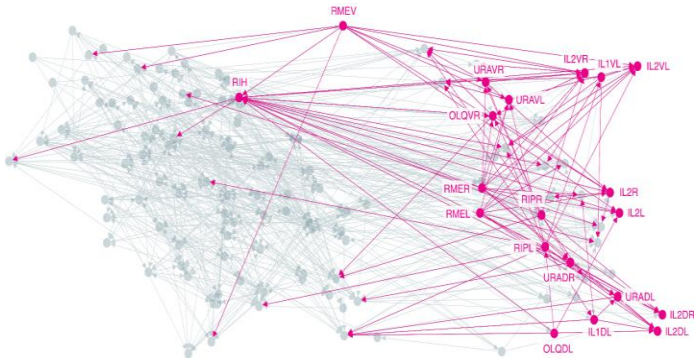
Biomedical networks



Information networks:
Web & citations



Internet



Networks of neurons

Why networks? Why now?

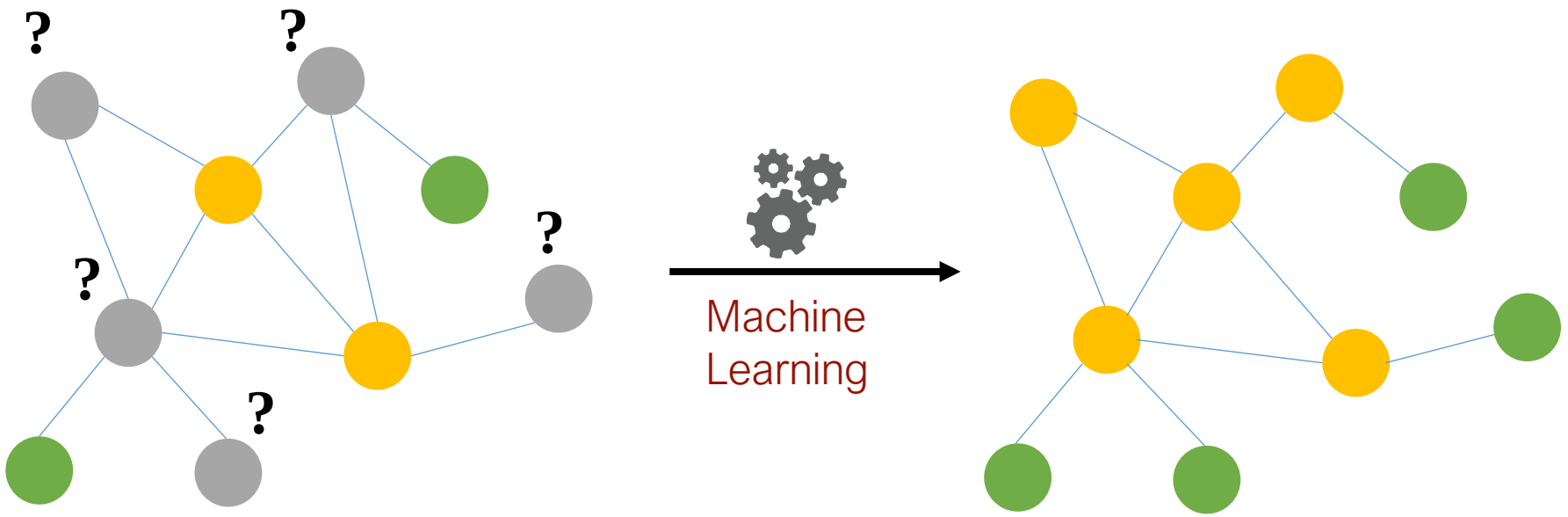
- Universal language for describing complex data
  - Networks from science, nature, and technology are more similar than one would expect
- Shared vocabulary between fields
  - Computer Science, Social science, Physics, Economics, Statistics, Biology
- Data availability (+computational challenges)
  - Web/mobile, bio, health, and medical

Machine learning with networks

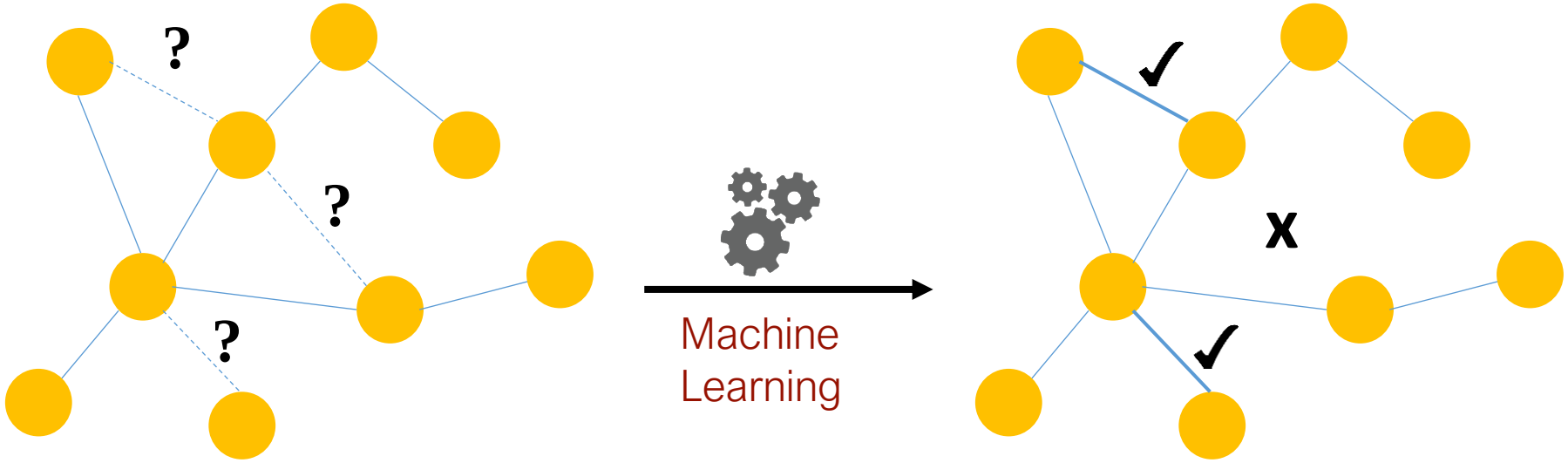
Classical ML tasks in networks:

- Node classification
  - Predict a type of a given node
- Link prediction
  - Predict whether two nodes are linked
- Community detection
  - Identify densely linked clusters of nodes
- Network similarity
  - How similar are two (sub)networks

Example: node classification

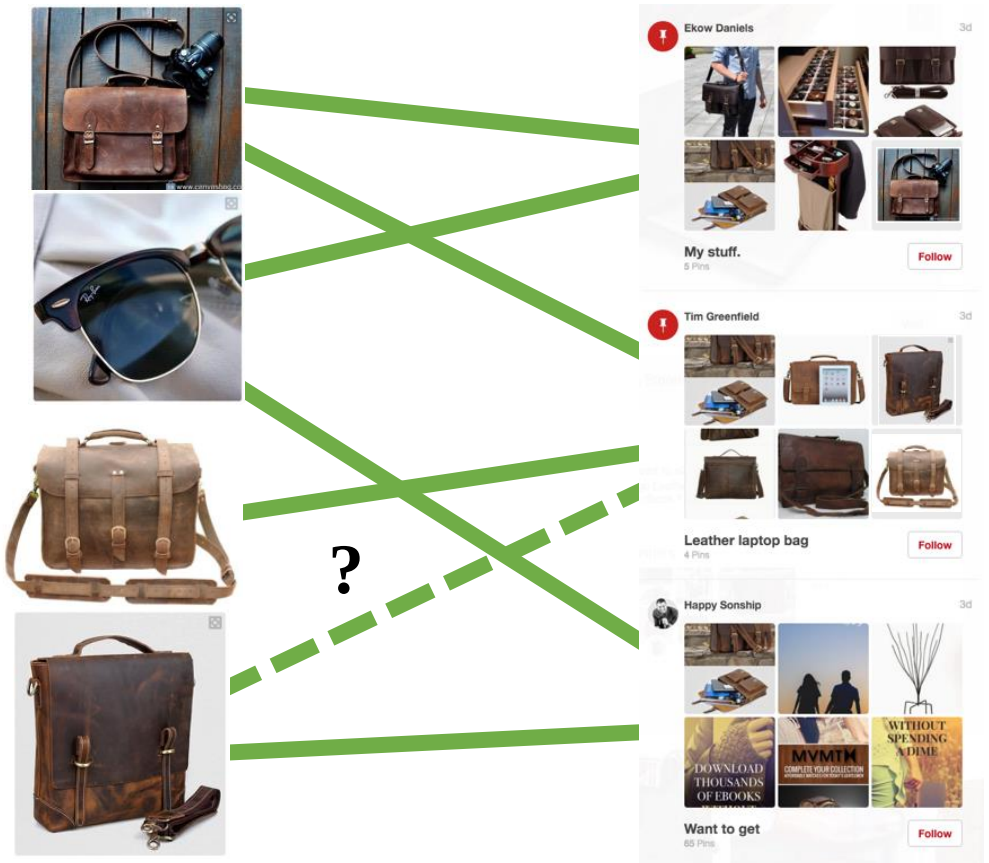


Example: link prediction



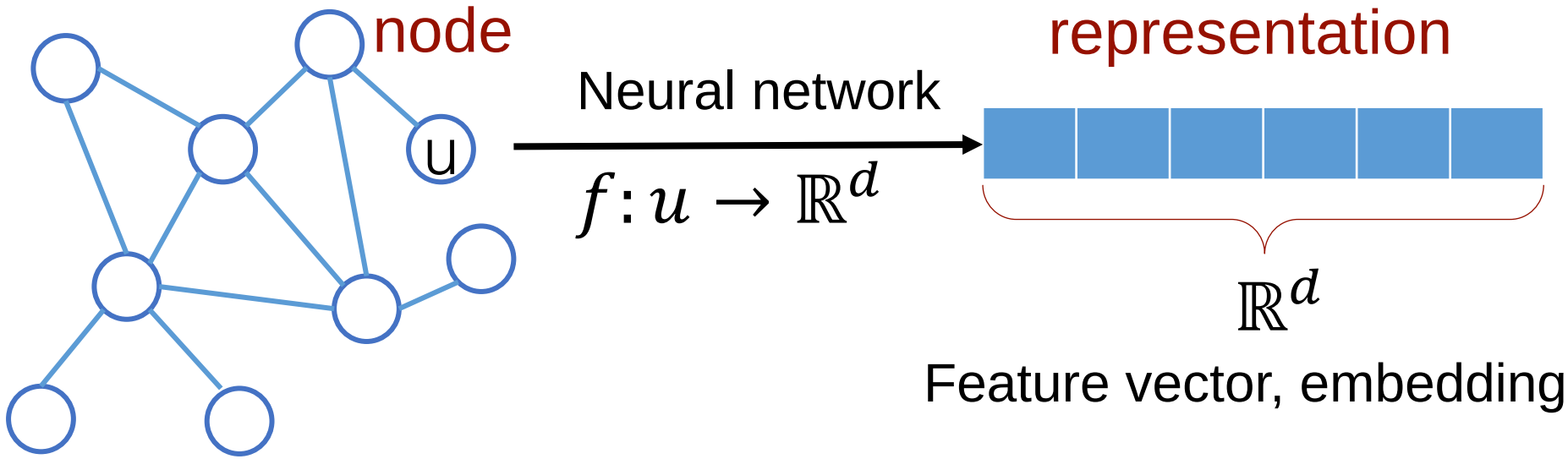
Example: link prediction

Content recommendation is link prediction!



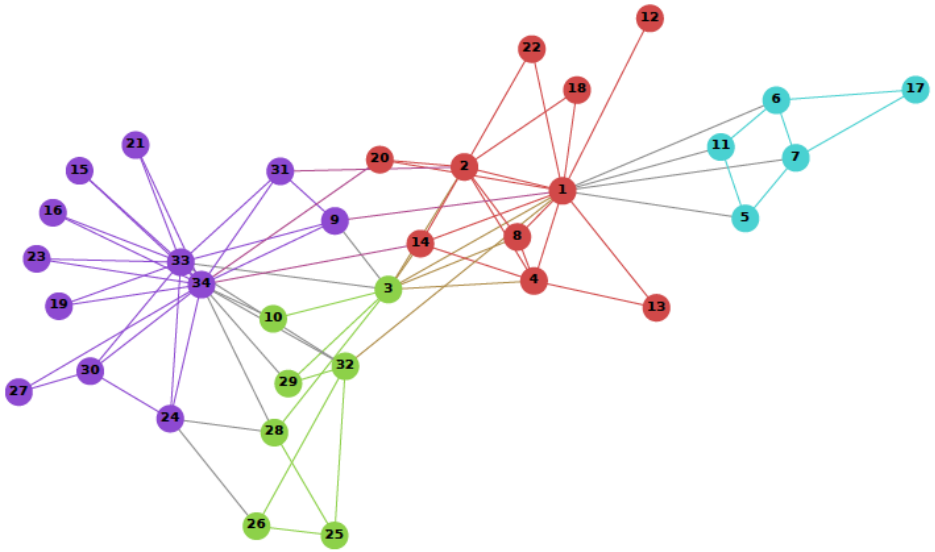
Representation learning of graphs

Goal: Map nodes to d-dimensional embeddings such that similar nodes in the network are embedded close together

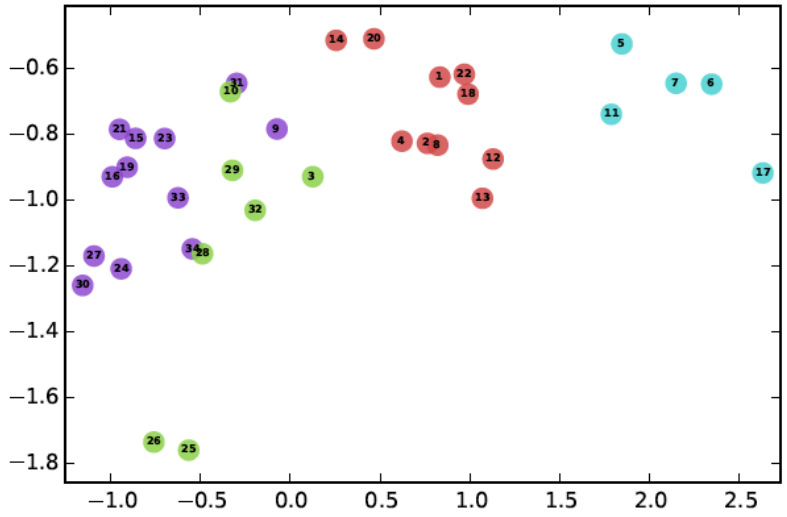


Example

- Zachary's Karate Club Network:



Input

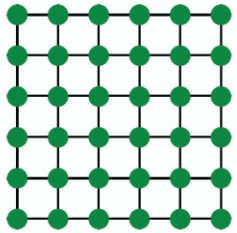
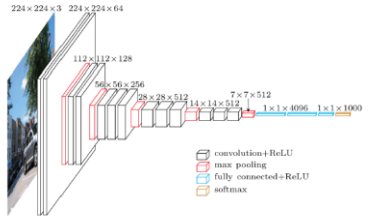


Output

Image from: Perozzi et al. 2014. DeepWalk: Online Learning of Social Representations. *KDD.*

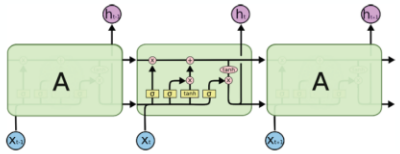
Why is graph learning hard?

- Modern deep learning toolbox is designed for simple sequences or grids.
  - CNNs for fixed-size images/grids….

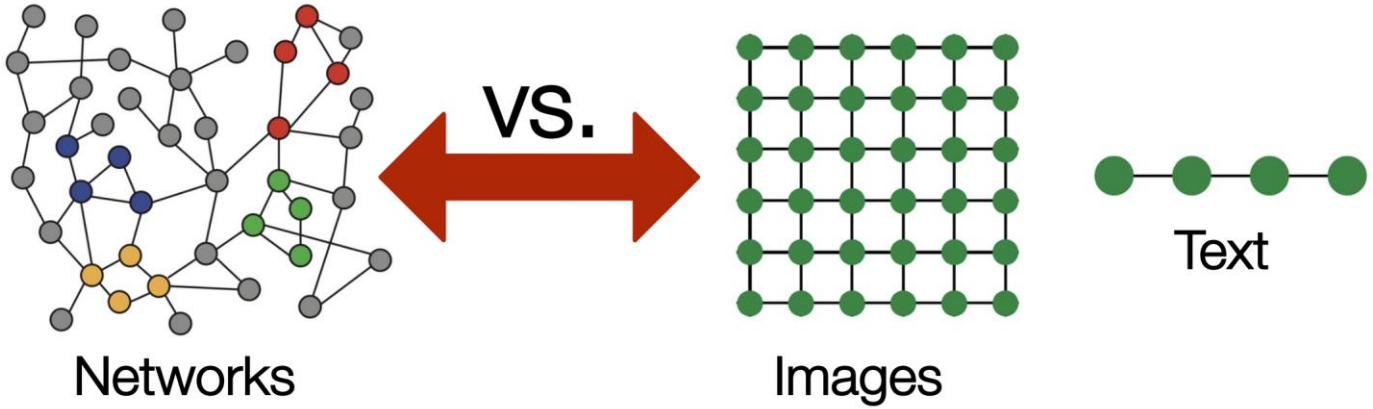
  - RNNs or word2vec for text/sequences…





Why is graph learning hard?

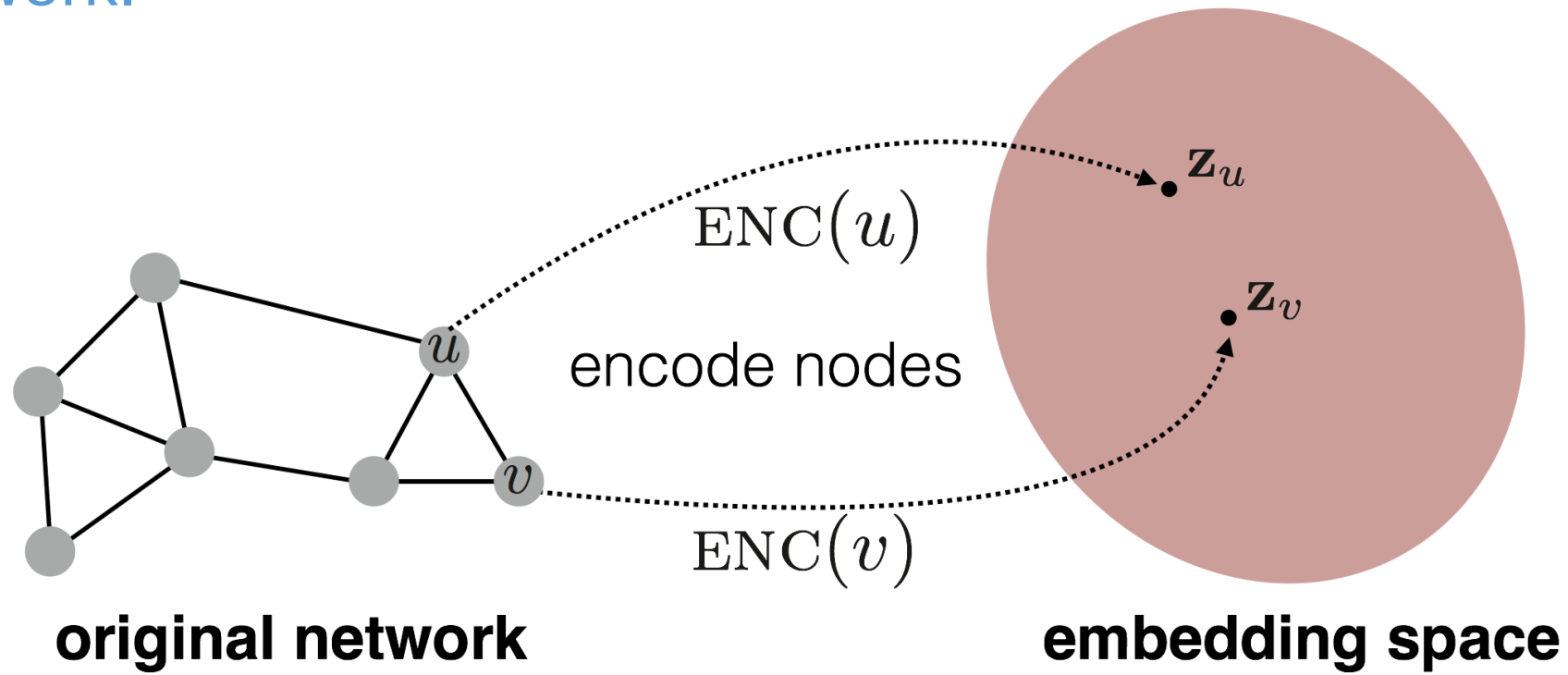
- But networks are far more complex!
  - Arbitrary size and complex topological structure (i.e., no spatial locality like grids)



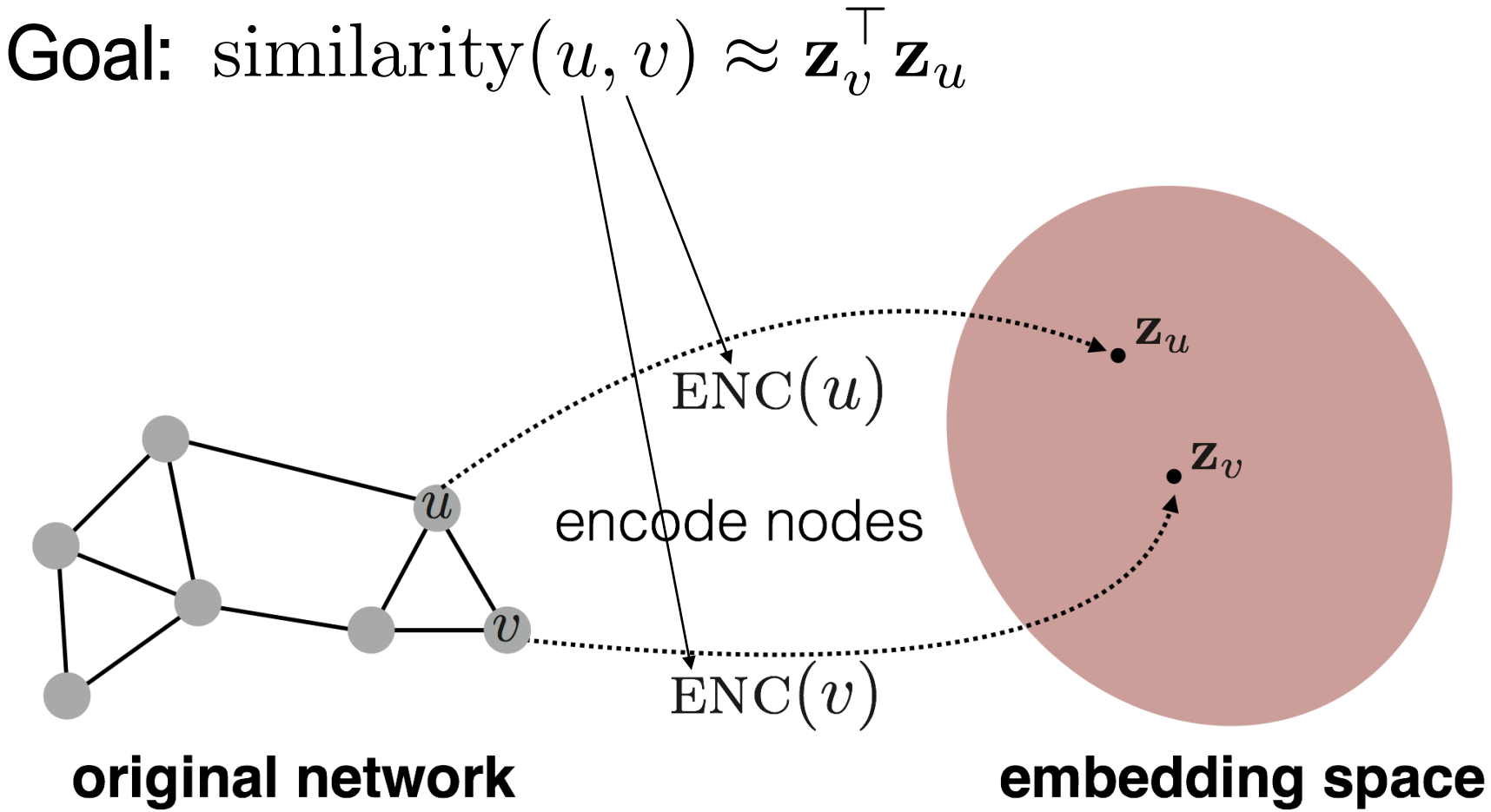
  - No fixed node ordering or reference point (i.e., the isomorphism problem)
  - Often dynamic and have multimodal features

Embedding nodes

- Goal is to encode nodes so that similarity in the embedding space (e.g., dot product) approximates similarity in the original network.

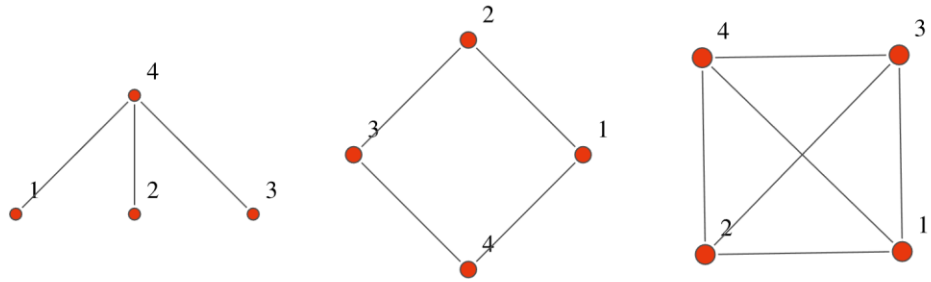


Embedding nodes



Setup for graph convolutional network (GCN)

- Assume we have a graph $\boldsymbol{G}$:
  - V is the vertex set.
  - $\boldsymbol{A}$ is the adjacency matrix (assume binary).
  - $\boldsymbol{X} \in \mathbf{R}^{m \times |V|}$ is a matrix of node features.
    - Social networks: User profile, User image
    - Biological networks: Gene expression profiles, gene functional information
    - When there is no node feature in the graph dataset:
      - Indicator vectors (one-hot encoding of a node)
      - Vector of constant 1: [1, 1, …, 1]
  - v: a node in V; $N(v)$: the set of neighbor of v

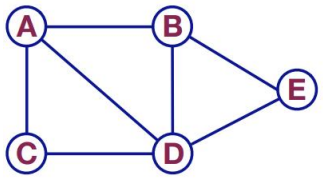


$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

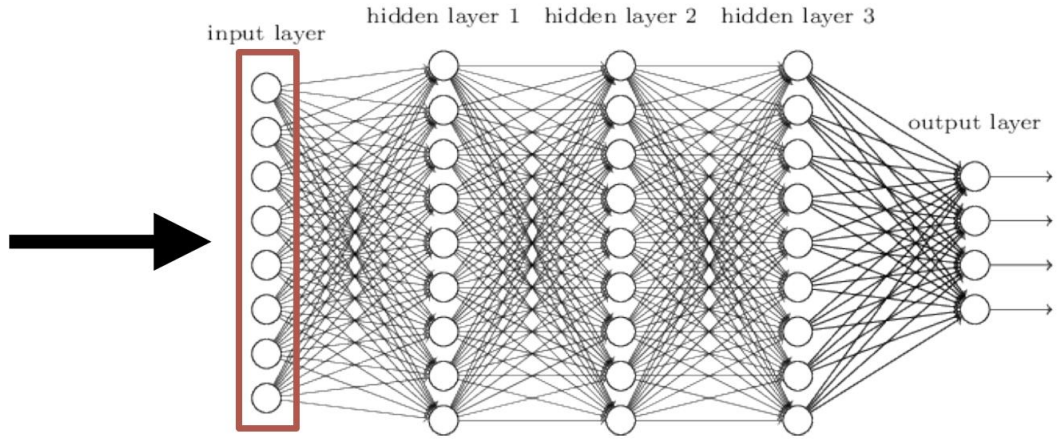
Example of adjacency matrix showing the edges in graphs

A naïve approach

- Join adjacency matrix and features
- Feed them into a deep neural network



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0

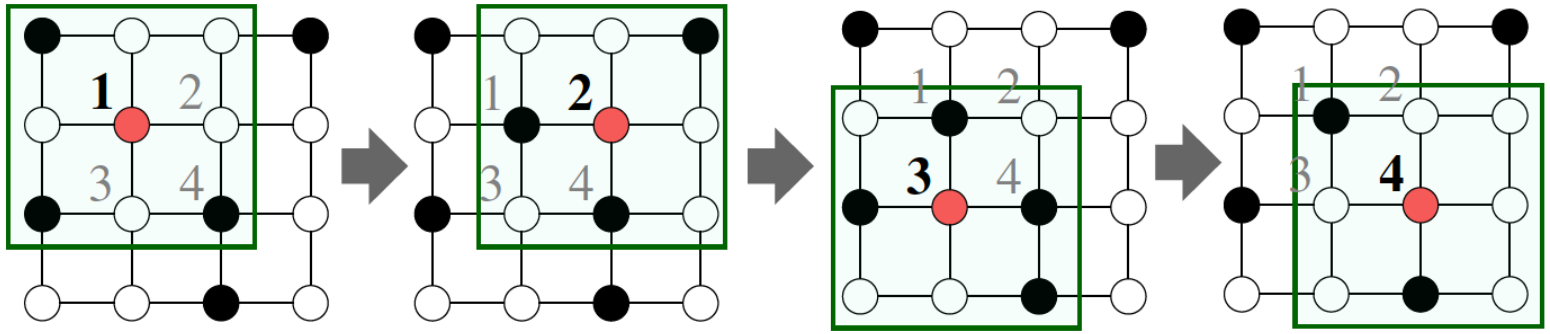


?

- Problems?
  - O(|V|) parameters are needed
  - Not applicable to graphs of different sizes
  - Sensitive to node ordering

Convolutional networks on images

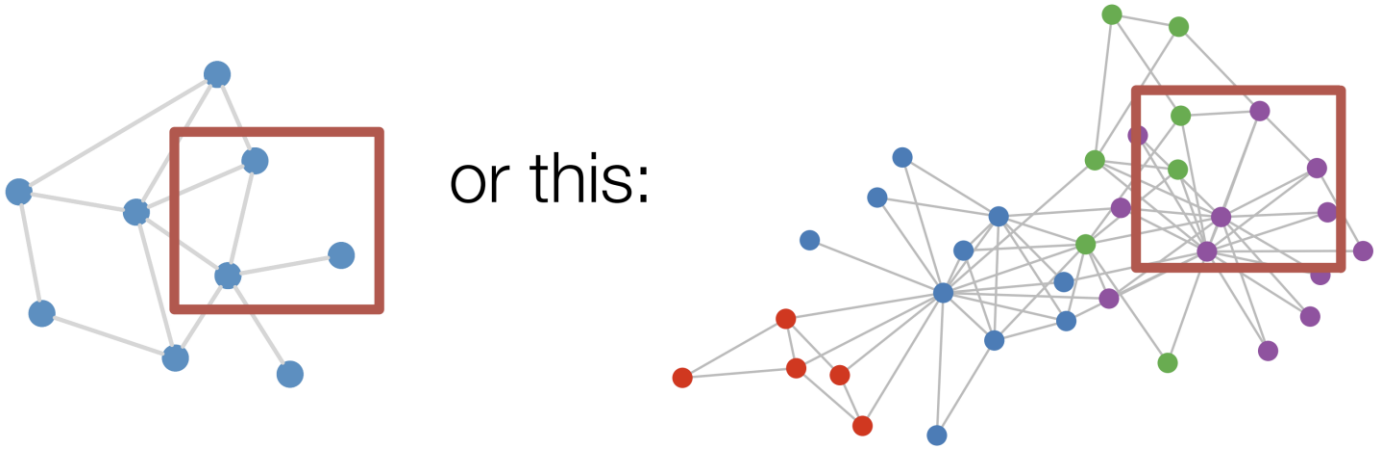
- Convolutional layer on an image



- Idea: pixels or features have local patterns that close pixels should be put together for recognition.

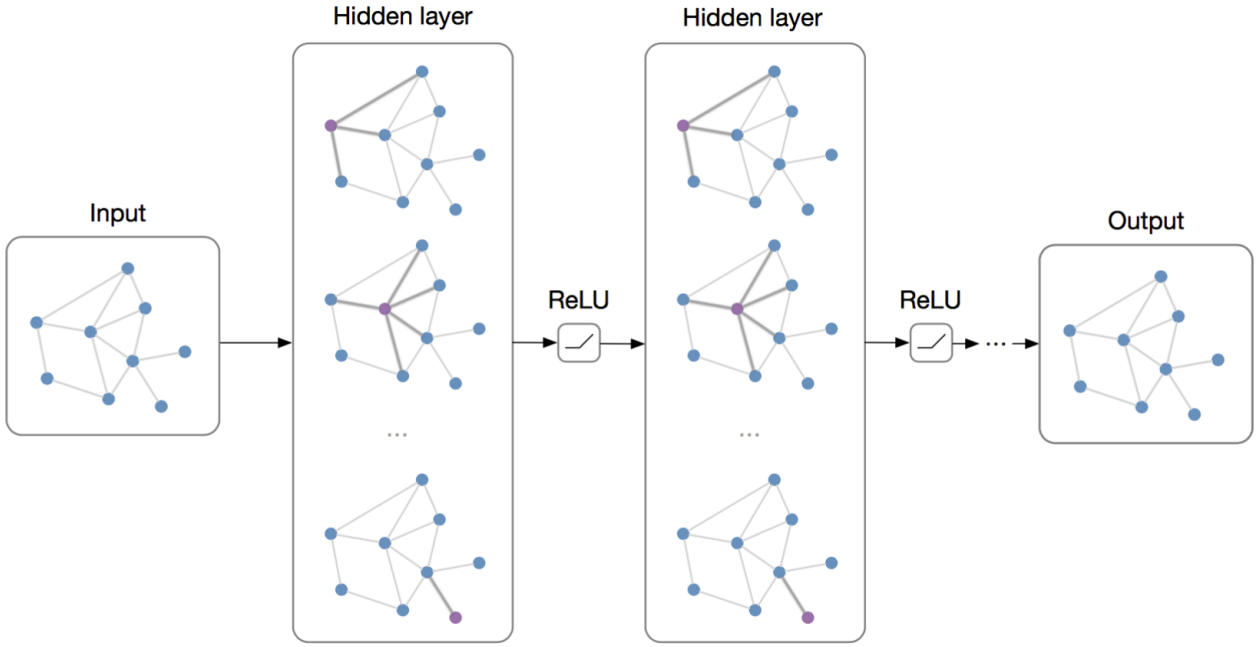
Can we make convolution on graphs?

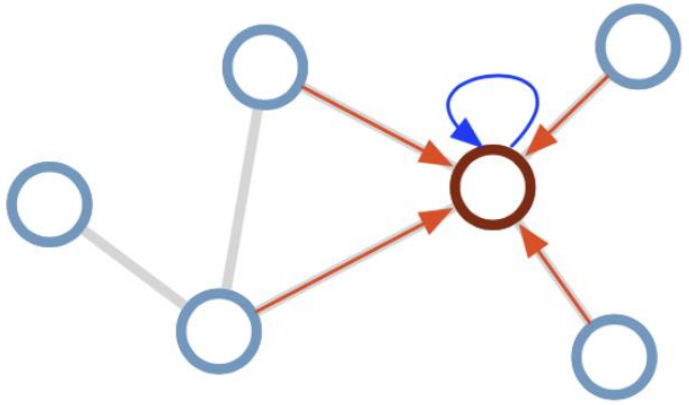
- Individual nodes are revealed by local neighbors (e.g., friends share the interest on the same sports)



- There is no fixed notion of locality or sliding window on the graph
- Graph is permutation invariant (location or order doesn't matter)

Convolution on graphs as message passing



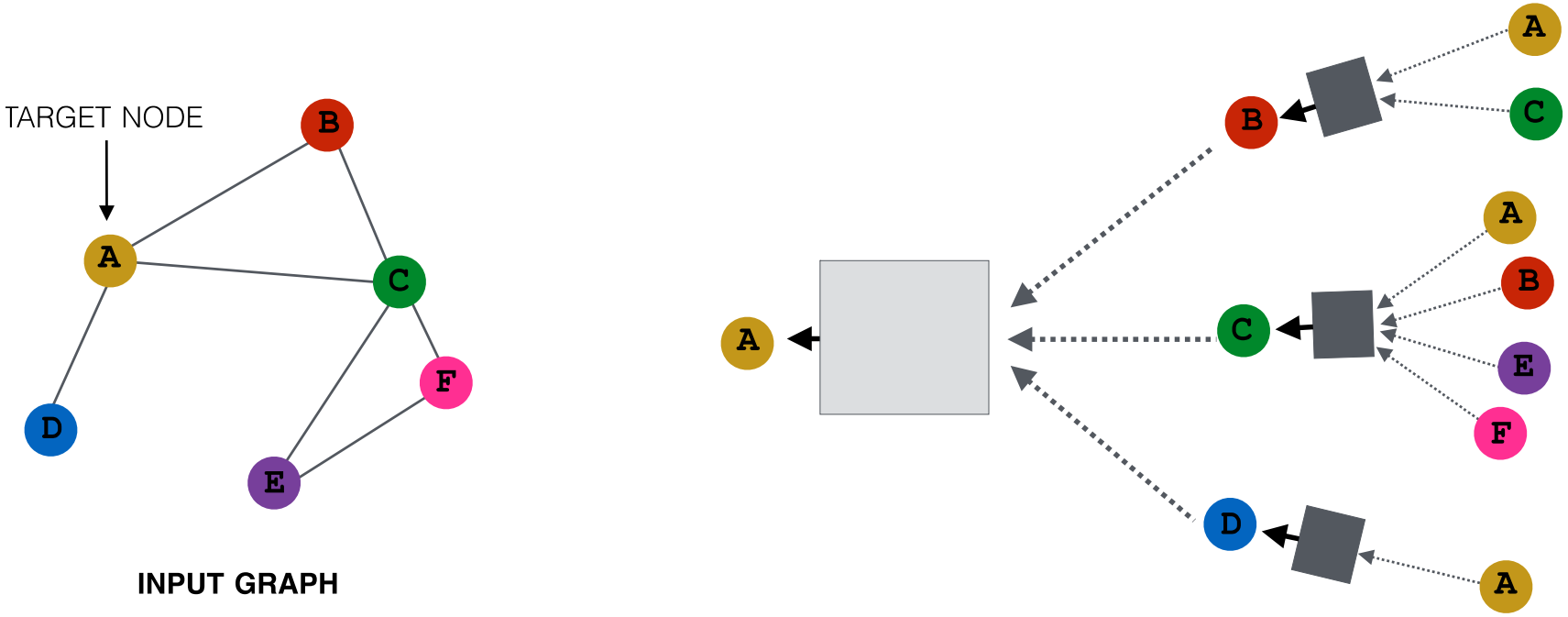


- For target node, pass massages from its neighbor nodes and aggregate information
- Make local “message passing” for every node
- Repeat the above steps for N iterations

- Transform “messages” h_i from neighbors $W_i h_i$
- Add them up $\sum_i W_i h_i$

Neighborhood aggregation

- **Key idea:** Each node defines a computation graph
  - To aggregate the representation/feature of neighboring nodes
  - Each edge in this graph is a transformation/aggregation function

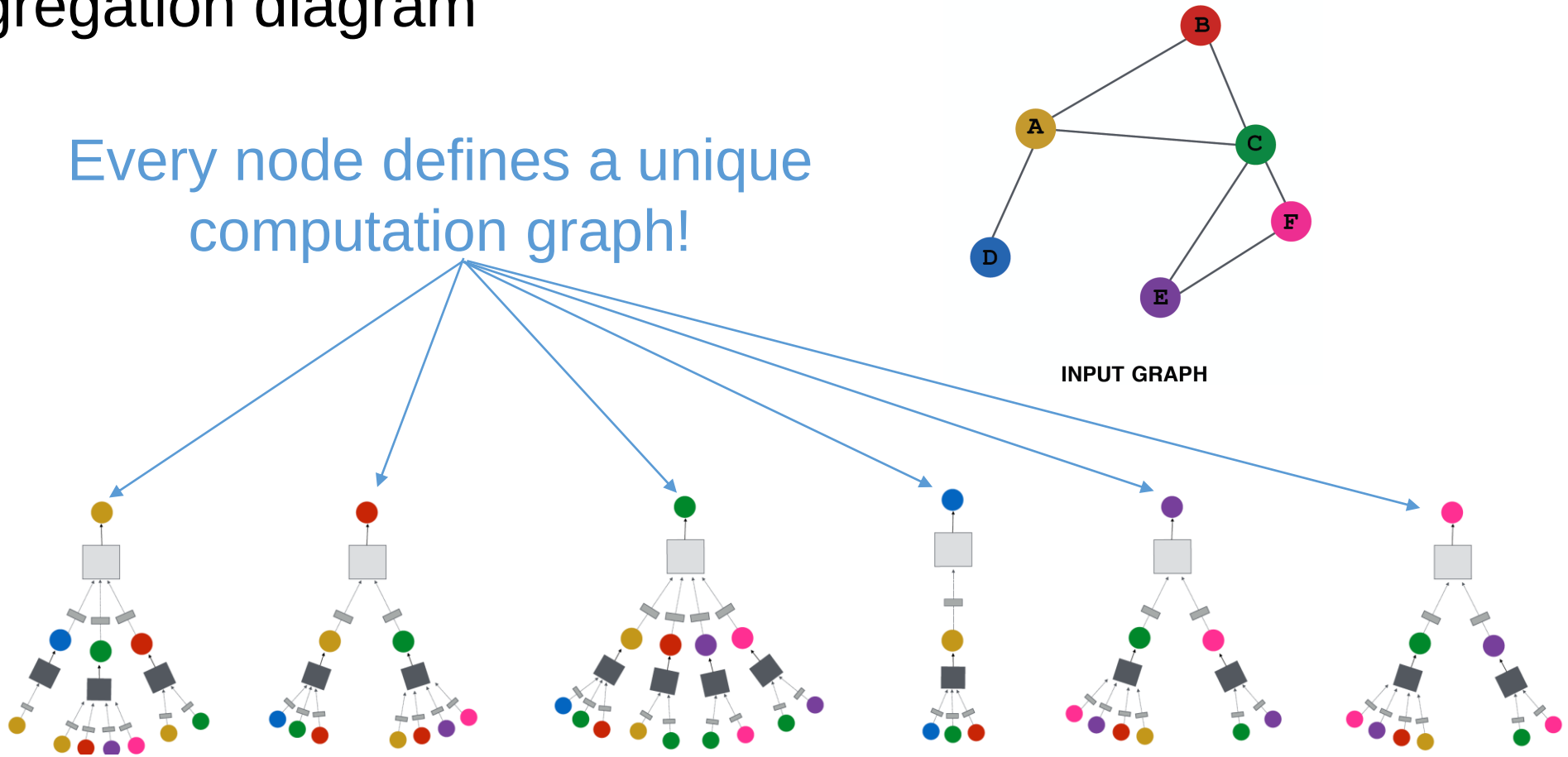


Scarselli et al. 2005. The Graph Neural Network Model. IEEE Transactions on Neural Networks.

Neighborhood aggregation

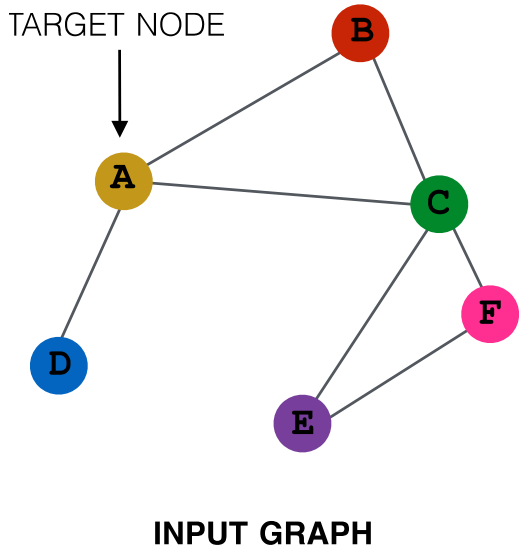
- Each node will be treated as a target node and has its own aggregation diagram

Every node defines a unique computation graph!

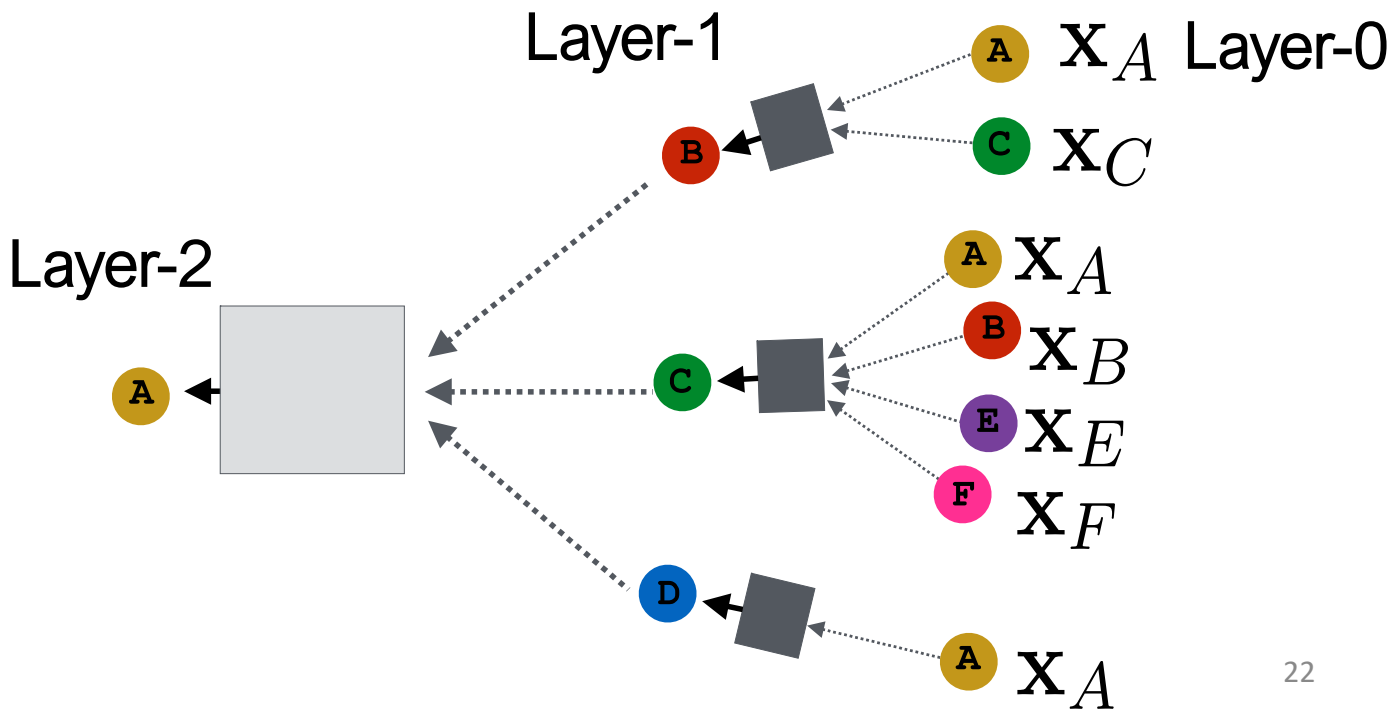


Neighborhood aggregation

- Nodes have embeddings at each layer.
- Model can be arbitrary depth.
- “layer-0” embedding of node u is its input feature, i.e. x_u.
- “layer-K” embedding gets information from nodes that are K hops away

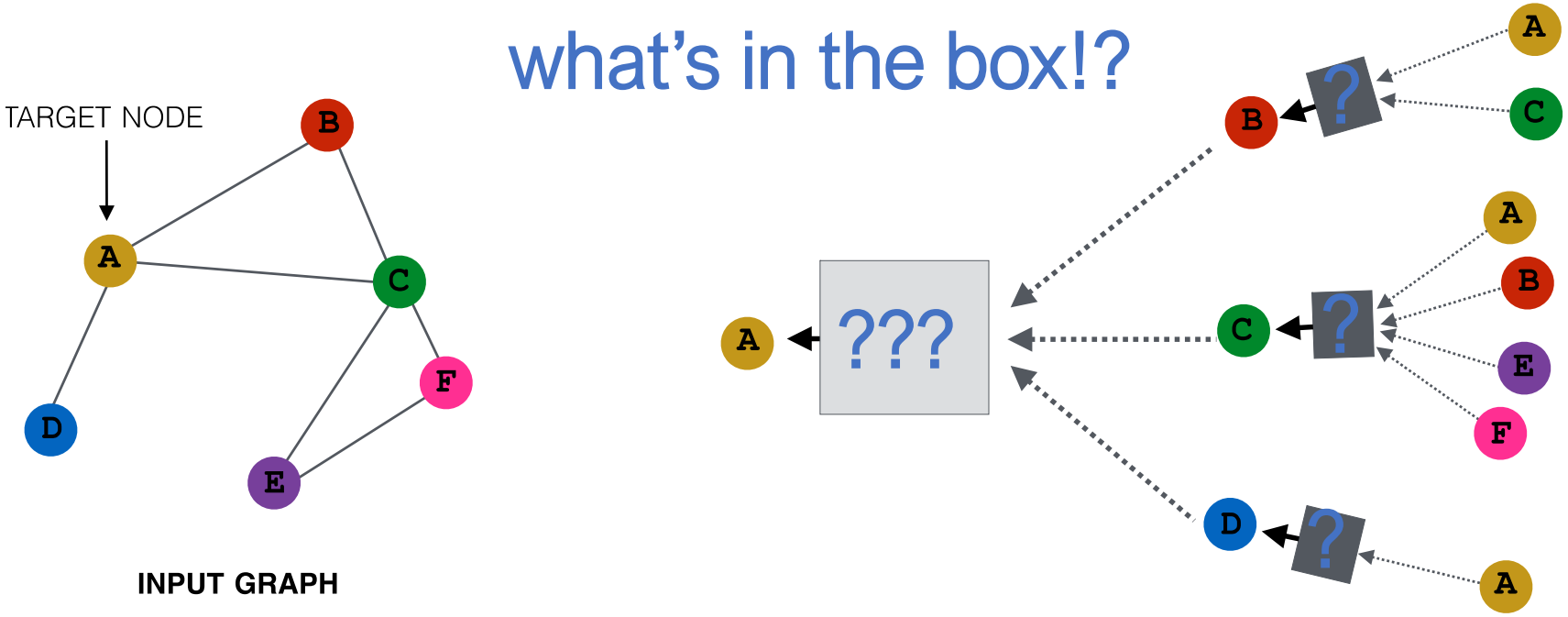


INPUT GRAPH



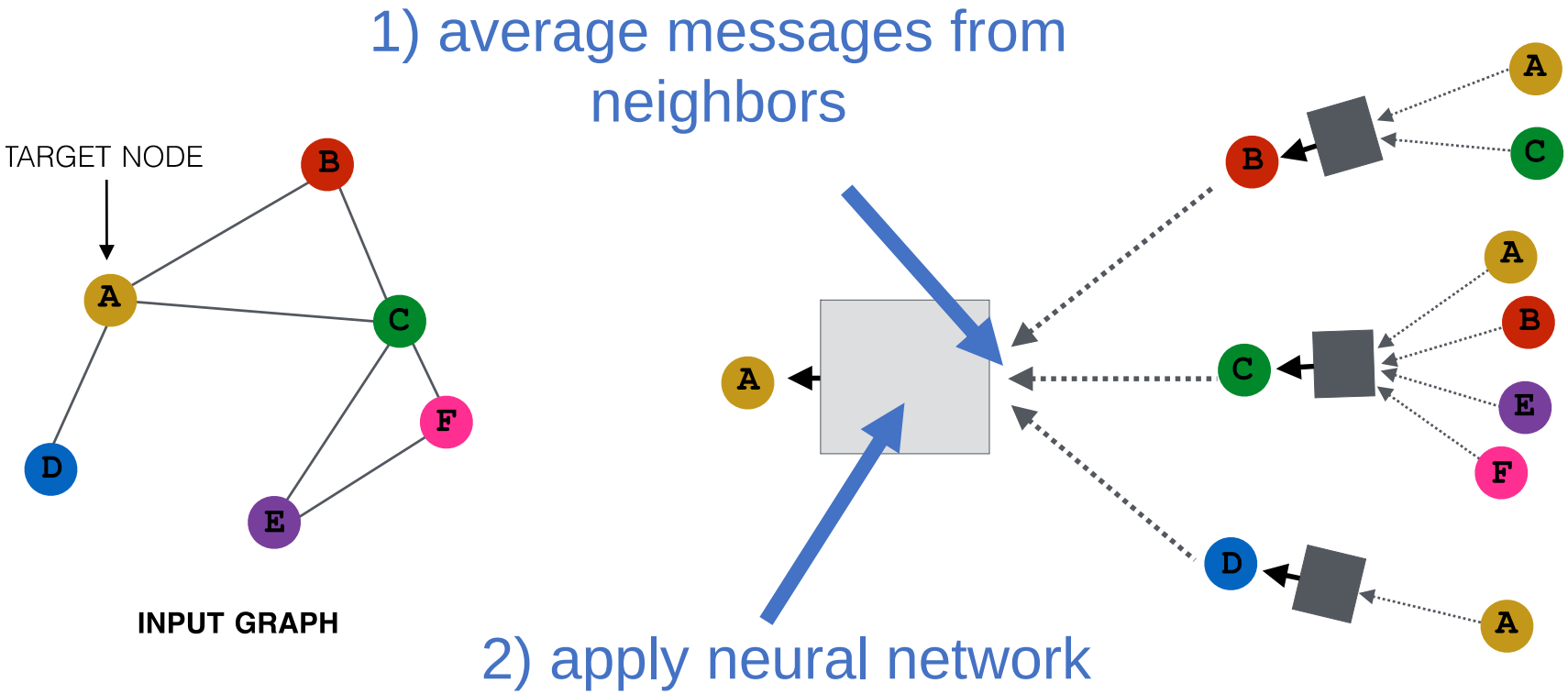
Neighborhood aggregation

- Key distinctions are in how different approaches aggregate information across the layers.



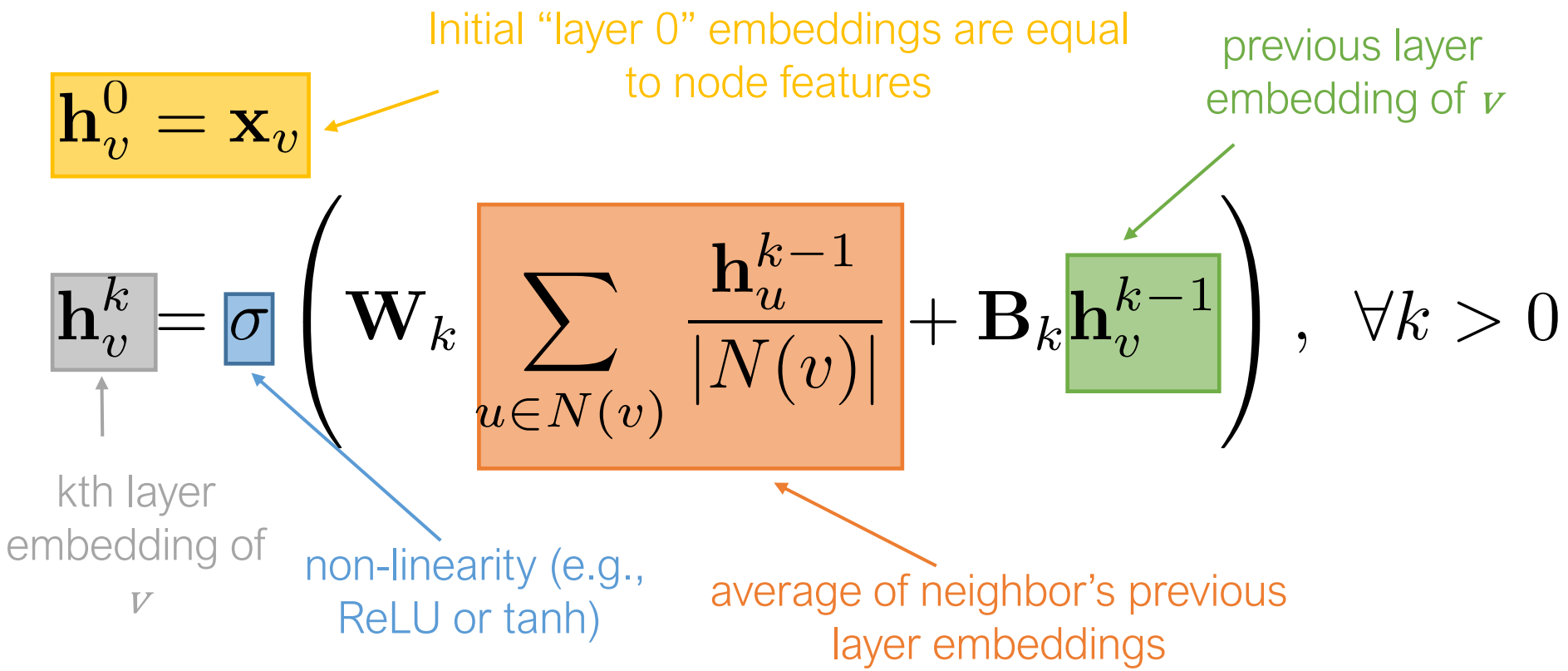
Neighborhood aggregation

- **Basic approach:** Average neighbor information and apply a neural network.



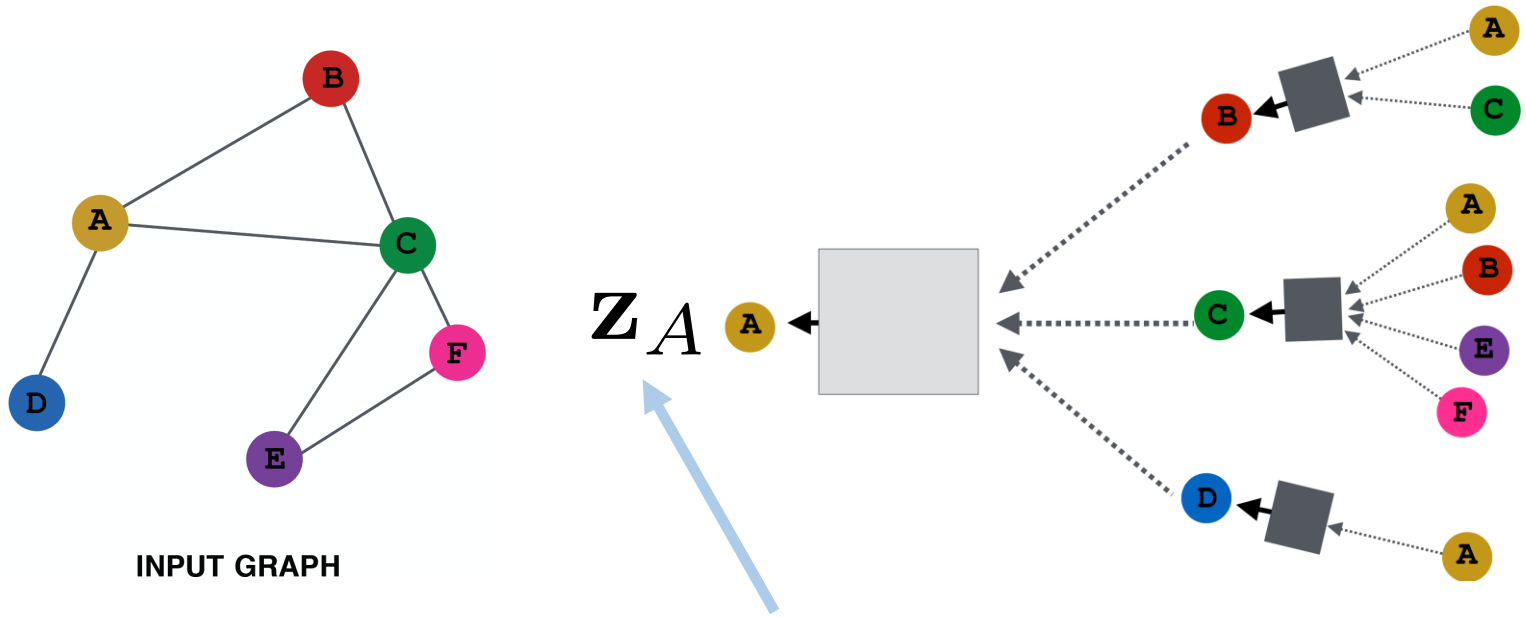
The math

- **Basic approach:** Average neighbor messages and apply a neural network.



Training the model

- How do we train the model to generate “high-quality” embeddings?



Need to define a loss function on the embeddings, $L(z_u)$!

Training the model

trainable matrices
(i.e., what we learn)

$$\mathbf{h}_v^0 = \mathbf{x}_v$$

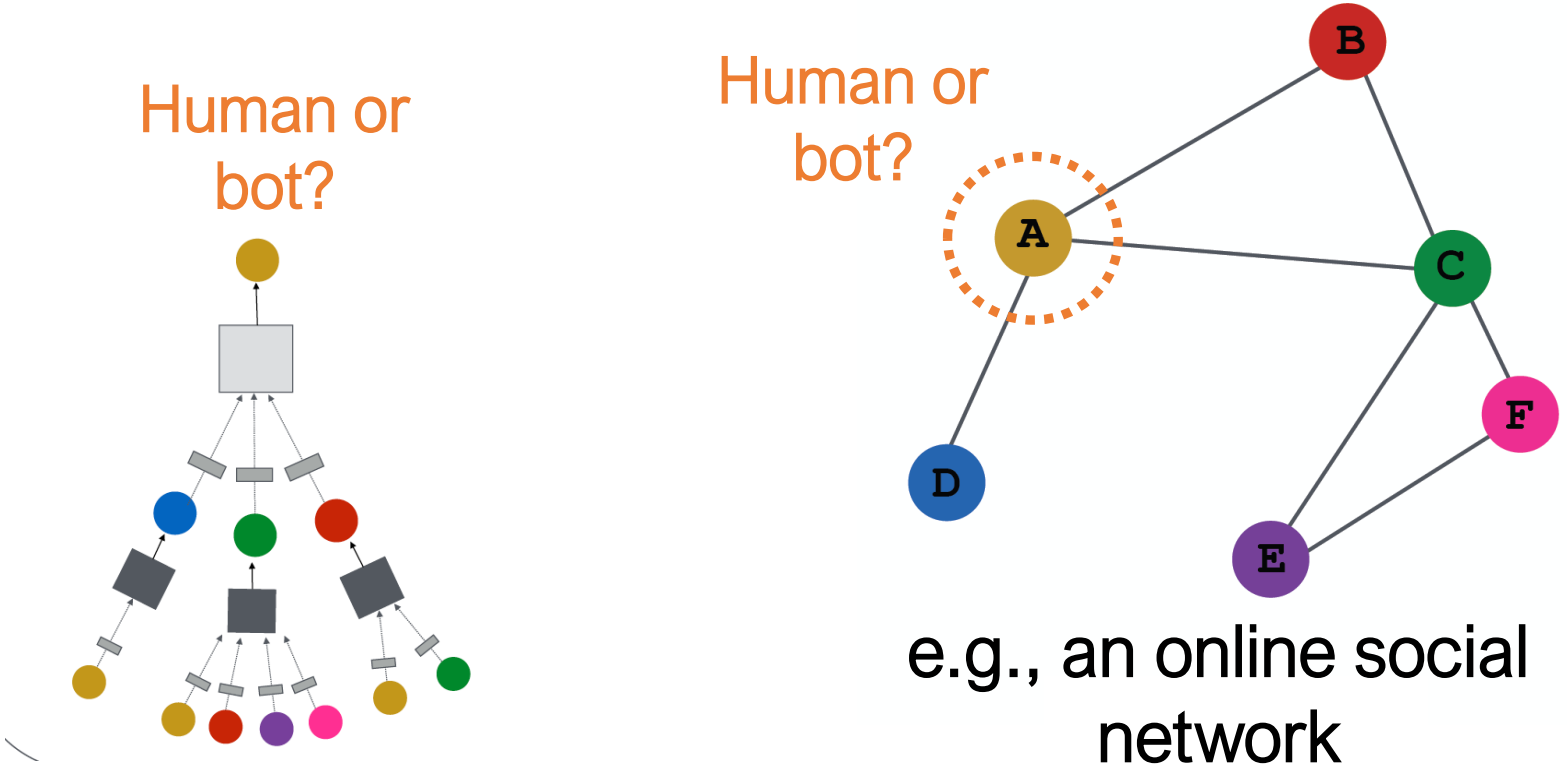
$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1}\right), \quad \forall k \in \{1, ..., K\}$$

$$\mathbf{z}_v = \mathbf{h}_v^K$$

- After K-layers of neighborhood aggregation, we get output embeddings for each node.
- **We can feed these embeddings into any loss function** and run stochastic gradient descent to train the aggregation parameters.

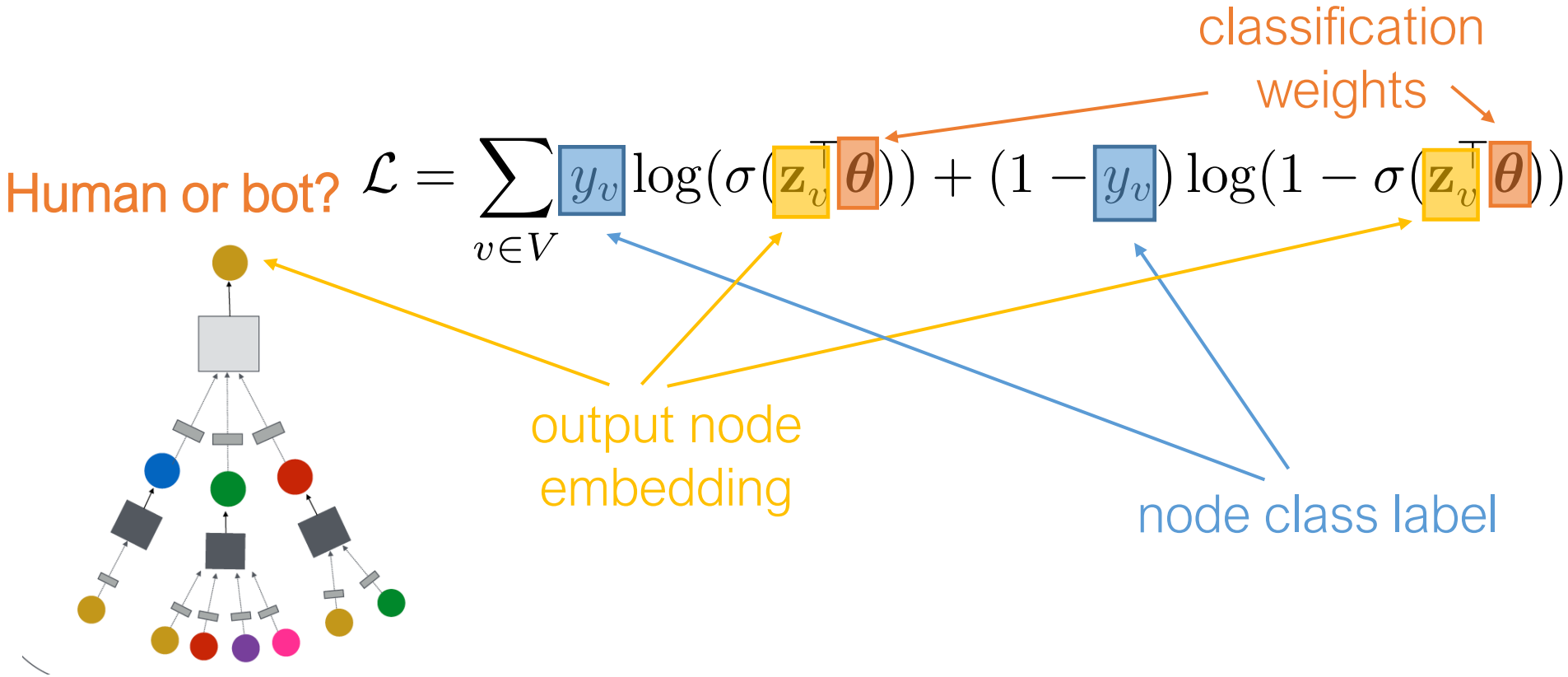
Training the model

- Ttrain the model for a supervised task (e.g., node classification):



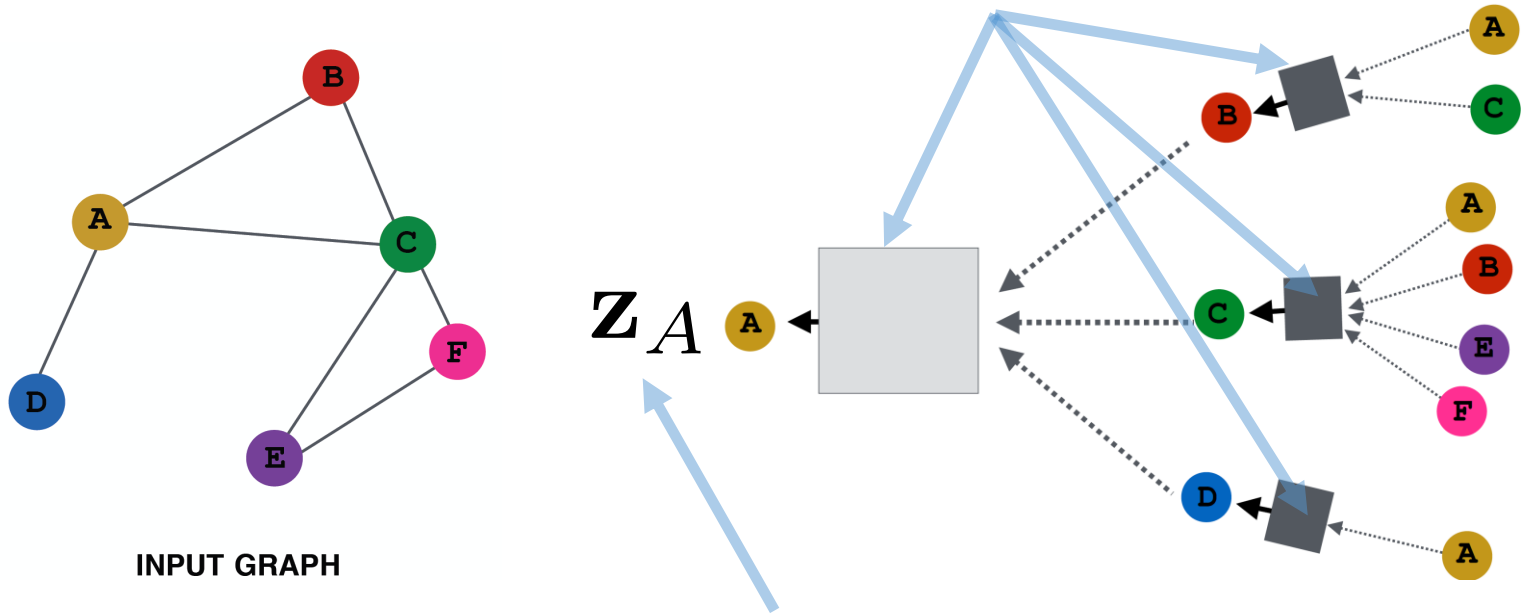
Training the model

- Train the model for a supervised task (e.g., node classification):



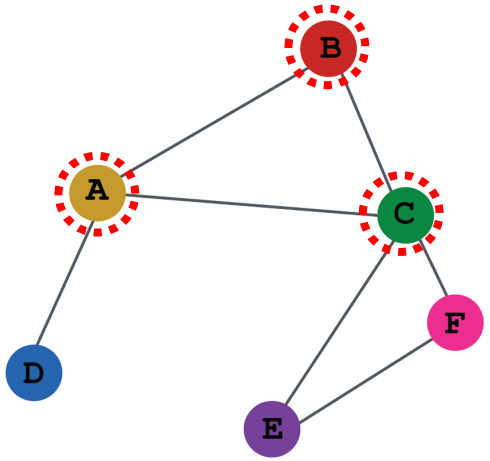
Overview of model design

1) Define a neighborhood aggregation function.



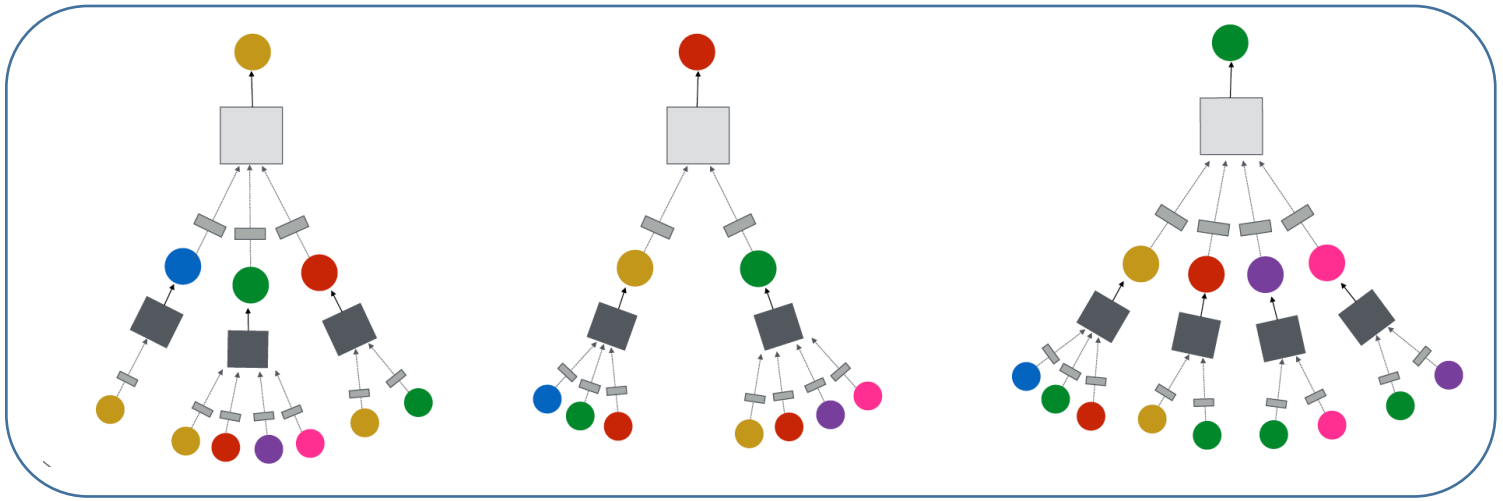
2) Define a loss function on the embeddings, $L(z_u)$

Overview of model design

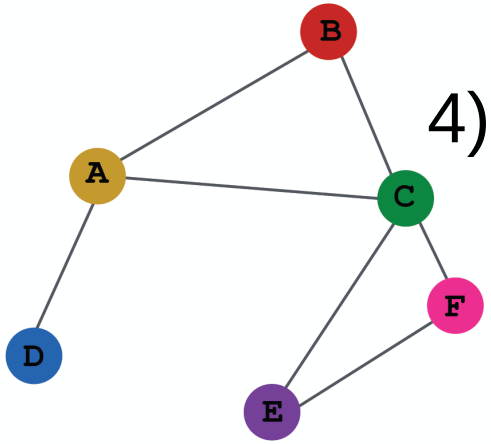


INPUT GRAPH

3) Train on a set of nodes, i.e., a batch of compute graphs



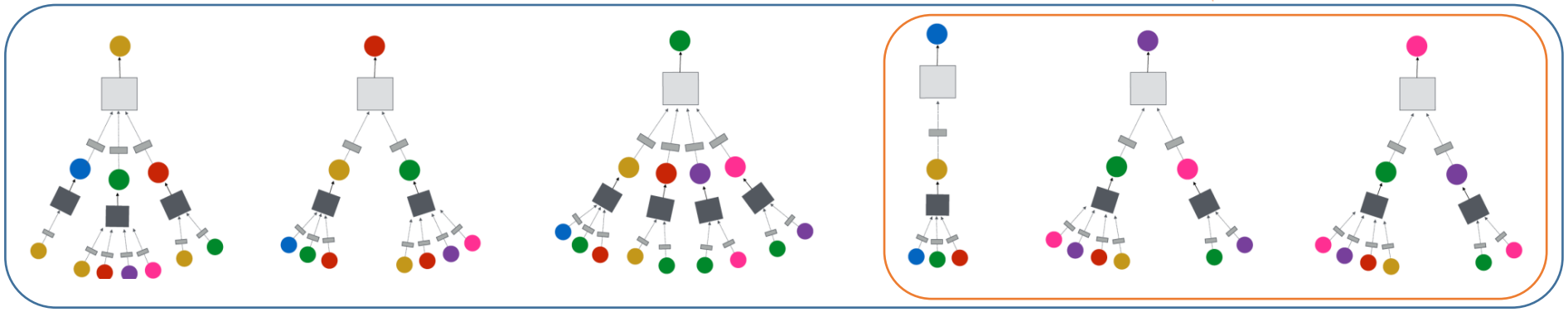
Overview of model design



INPUT GRAPH

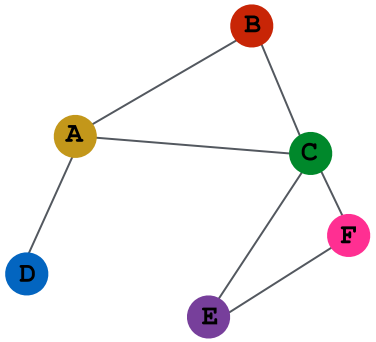
4) Generate embeddings for nodes as needed

Even for nodes we never trained on!!!!

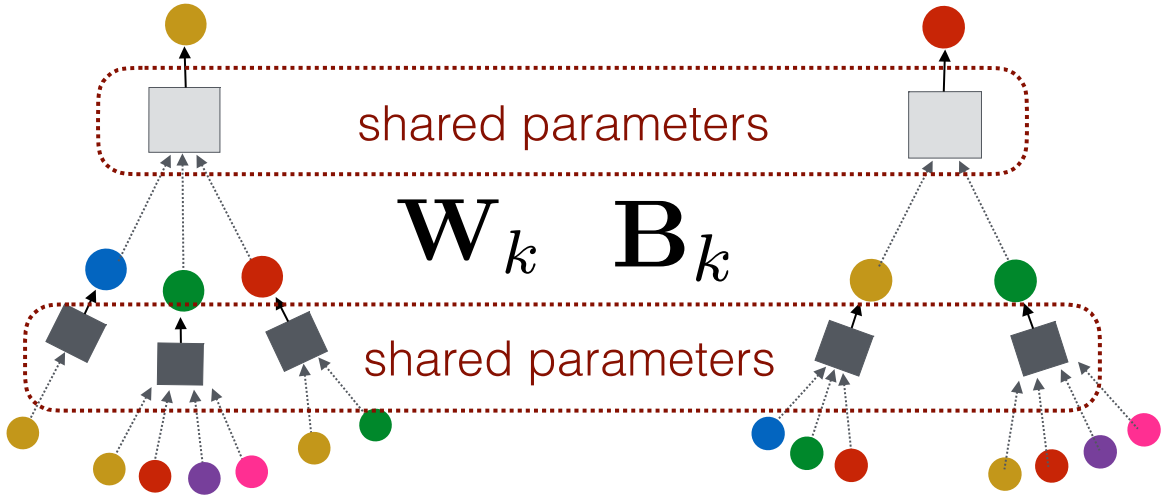


Inductive capability

- The same aggregation parameters are shared for all nodes.
- The number of model parameters is sublinear in $|V|$ and we can generalize to unseen nodes!

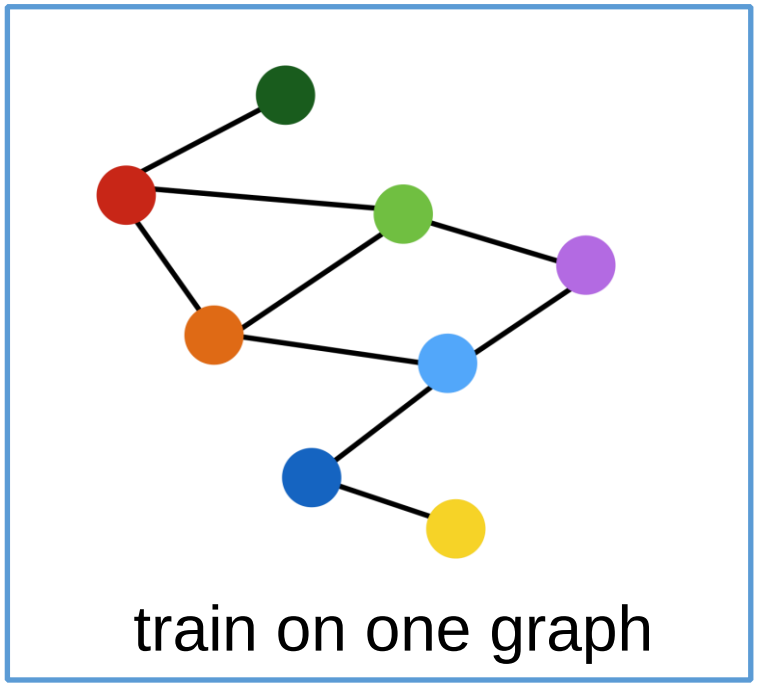


INPUT GRAPH

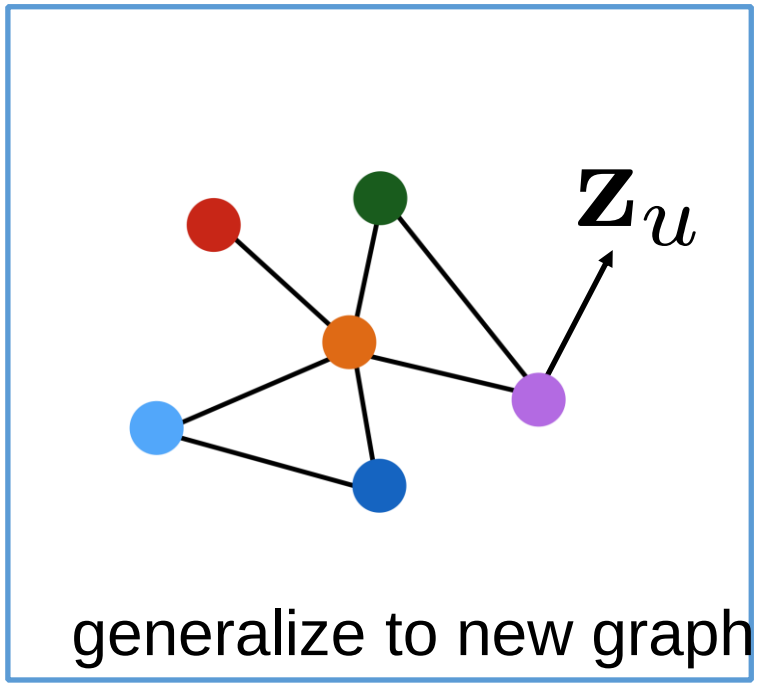


Compute graph for node A **Compute graph for node B**

Inductive capability



train on one graph

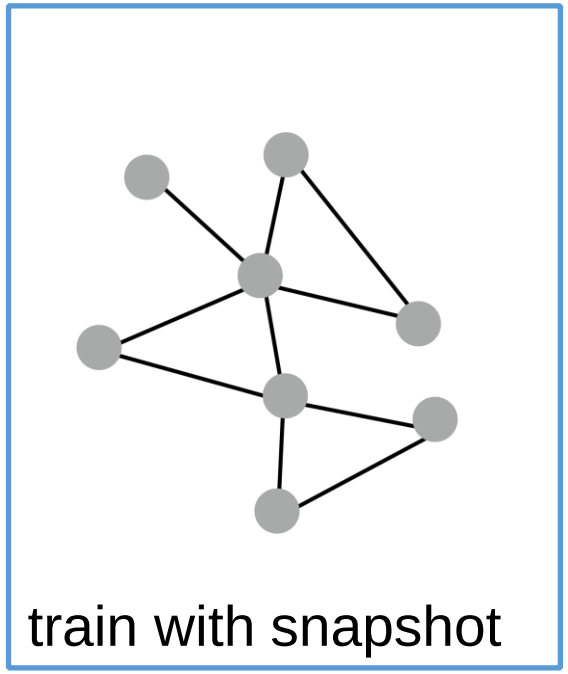


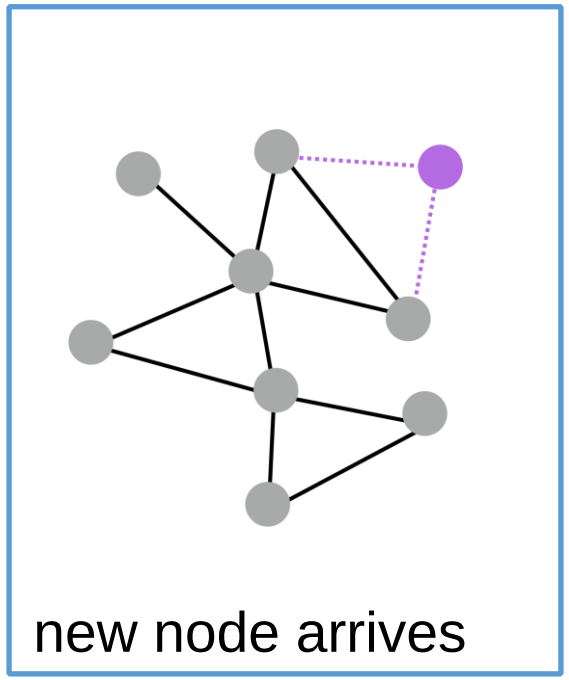
generalize to new graph

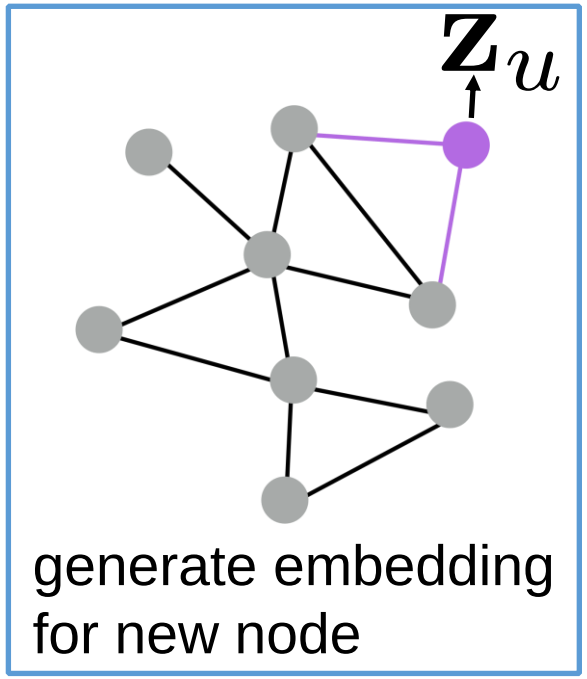
Inductive node embedding → generalize to entirely unseen graphs

e.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Inductive capability







Many application settings constantly encounter previously unseen nodes.

e.g., Reddit, YouTube, GoogleScholar, ….

Need to generate new embeddings “on the fly”

Graph convolutional networks

- Kipf et al.'s **Graph Convolutional Networks (GCNs)** are a slight variation on the neighborhood aggregation idea:

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

Graph convolutional networks

Basic Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1}\right)$$

VS.

GCN Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

same matrix for self and neighbor embeddings

per-neighbor normalization

Graph convolutional networks

- Empirically, they found this configuration to give the best results.
  - More parameter sharing.
  - Down-weights high degree neighbors.

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

use the same transformation matrix for self and neighbor embeddings

instead of simple average, normalization varies across neighbors

Batch implementation

- Can be efficiently implemented using sparse batch operations:

$$\mathbf{H}^{(k+1)} = \sigma\left(\mathbf{D}^{-\frac{1}{2}}\tilde{\mathbf{A}}\mathbf{D}^{-\frac{1}{2}}\mathbf{H}^{(k)}\mathbf{W}_k\right)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$

$$\mathbf{D}_{ii} = \sum_j \mathbf{A}_{i,j}$$

- $O(|E|)$ time complexity overall.